

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



BİL-304 BİLGİSAYAR MİMARİSİ Dr. Öğr. Üyesi Fatih GÖKÇE

8. Hafta

Bölüm / Hafta Hedefleri

- **Komut Seti Mimarisinin 3. Kısımını anlatacağımız bu derste aşağıdaki konu başlıklarının öğrenilmesi hedeflenmektedir:**
 - Dallanma, Şartlı Taşıma Komutları,
 - Yığın işlemleri,
 - Alt program çağırma ve geri dönme,
 - Y86-64 kod örnekleri,
 - RISC ve CISC mimarileri

Bu ders notundaki slaytlar ders kitabına ait sunum dosyalarından adapte edilmiştir:

<http://csapp.cs.cmu.edu/>

Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

Dallanma (Jump) Komutları

Jump (Şarta bağlı olarak)

jxx Hedef	7	fn	Hedef
-----------	---	----	-------

- Genel olarak “jxx” şeklinde ifade edilirler
- Kodlamaları sadece “fonksiyon kodu” (fn) kısmında farklıdır
- Durum kodlarının (condition codes) değerlerine bağlıdır
- x86-64'daki dallanma komutlarına benzerler
- Hedef adresin tamamı komut içerisinde kodlanır
 - x86-64'teki PC değerine göre bağlı olarak yapılan adreslemeden farklıdır

Dallanma (Jump) Komutları

Şartsız Dallan

jmp Dest 7 0 Dest

Küçük veya Eşitse Dallan

jle Dest 7 1 Dest

Küçükse Dallan

j1 Dest 7 2 Dest

Eşitse Dallan

je Dest 7 3 Dest

Eşit Değilse Dallan

jne Dest 7 4 Dest

Büyük veya Eşitse Dallan

jge Dest 7 5 Dest

Büyükse Dallan

jg Dest 7 6 Dest

Dallanma

jX Komutları

- Durum kodlarına (condition codes) bağlı olarak kodun farklı bir kısmına dallan

jX	Durum	Tanım
jmp	1	Şartsız
je	ZF	Eşit / Sıfır
jne	$\sim ZF$	Eşit değil / Sıfır değil
jg	$\sim (SF \wedge OF) \ \& \ \sim ZF$	Büyükse (İşaretli(Signed))
jge	$\sim (SF \wedge OF)$	Büyük veya Eşitse (İşaretli(Signed))
jl	$(SF \wedge OF)$	Küçükse (İşaretli(Signed))
jle	$(SF \wedge OF) \ \ ZF$	Küçük veya Eşitse (İşaretli(Signed))

Şartlı Taşıma (Move) Komutları

Şartsız Taşı

`rrmovq rA, rB`

2	0	rA	rB
---	---	----	----

Küçük veya Eşitse Taşı

`cmovle rA, rB`

2	1	rA	rB
---	---	----	----

Küçükse Taşı

`cmovl rA, rB`

2	2	rA	rB
---	---	----	----

Eşitse Taşı

`cmove rA, rB`

2	3	rA	rB
---	---	----	----

Eşit Değilse Taşı

`cmovne rA, rB`

2	4	rA	rB
---	---	----	----

Büyük veya Eşitse Taşı

`cmovge rA, rB`

2	5	rA	rB
---	---	----	----

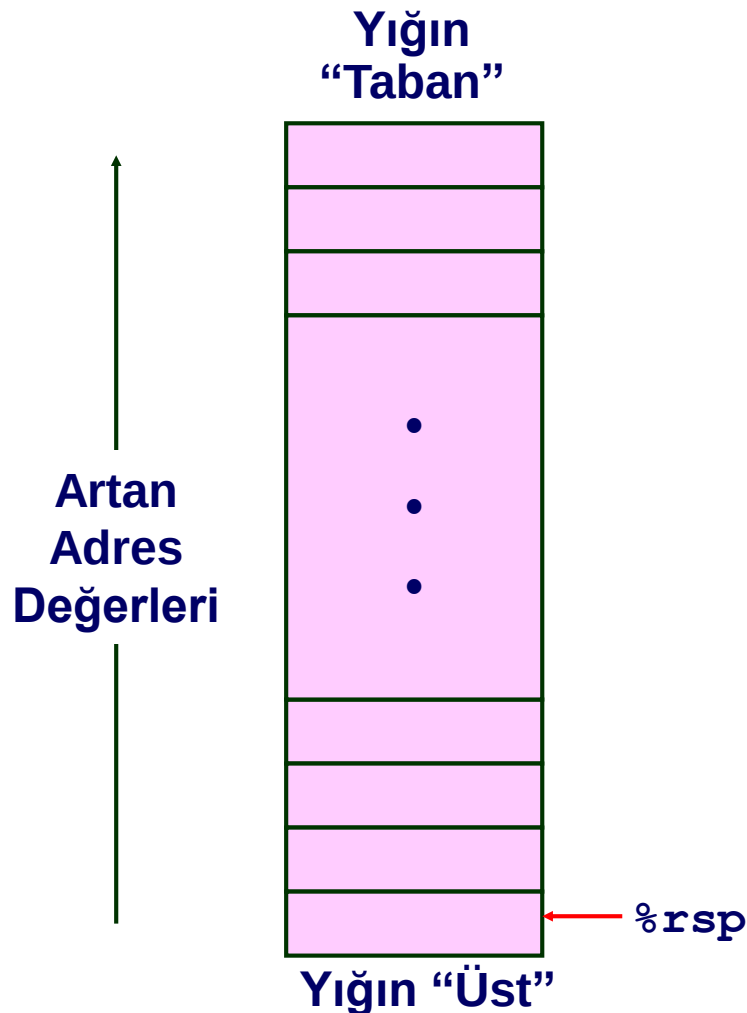
Büyükse Taşı

`cmovg rA, rB`

2	6	rA	rB
---	---	----	----

- Genel olarak “`cmovXX`” şeklinde ifade edilirler
- Kodlamaları sadece “fonksiyon kodu” (fn) kısmında farklıdır
- Durum kodlarının (condition codes) değerlerine bağlıdırlar
- Kaydediciden kaydediciye taşıma komutunun (`rrmovq`) türevidirler
 - (Bir şarta bağlı olarak) bir değeri kaynaktan hedefe kopyalarlar

Y86-64 Program Yığını (Stack)



- Program verisini tutan hafıza alanı
- Y86-64'te (ve X86-64'te) alt program çağırma sırasında kullanılır
- `%rsp` yığının en üstünü gösterir, yığın işaretçisi (stack pointer) olarak adlandırılır
 - En üst yığın elemanının adresini içerir
- Yığın düşük adrese doğru genişler
 - En üst eleman yığında en düşük adrese sahiptir
 - Push işlemi sırasında, önce yığın işaretçisi azaltılmalıdır
 - Pop işleminden sonra yığın işaretçisi artırılır

Yığın (Stack) İşlemleri

pushq rA

A	0	rA	F
---	---	----	---

- `%rsp` 'yi 8 azalt
- `rA`'daki sözcüğü `%rsp` 'deki değer ile gösterilen hafıza adresine kaydet
- x86-64'dakine benzer şekilde çalışır

popq rA

B	0	rA	F
---	---	----	---

- `%rsp` 'deki değer ile gösterilen hafıza adresindeki sözcüğü oku
- Okunan sözcüğü `rA`'ya kaydet
- `%rsp` 'yi 8 arttır
- x86-64'dakine benzer şekilde çalışır

Altprogram (Subroutine) çağırma(Call) ve geri dönme(Return)

`call Dest`

8 0

Hedef

- Bir sonraki komutun adresini yığına it (Push)
- Hedef adresindeki komutu işletmeye başla
- x86-64'dakine benzer şekilde çalışır

`ret`

9 0

- Yığından değeri çek
- Çekilden değeri bir sonraki komut adresi olarak kullan
- x86-64'dakine benzer şekilde çalışır

Diğer komutlar

`nop`

1	0
---	---

- Hiçbir şey yapma

`halt`

0	0
---	---

- Komutları işletmeyi durdur
- x86-64 benzer komutlara sahiptir, fakat kullanıcı modunda çalıştırılmazlar
- Burada işlemci simülatörünü durdurmak için kullanılmaktadır.
- 00 şeklinde kodlandığı için, programın 0 değeri atanmış, geçersiz hafıza adreslerinden komut işletmeye çalışması durumunda durmasını garanti eder.

Durum koşulları

Anımsatıcı (Mnemonic)	Kod
AOK	1

- Normal çalışma

Anımsatıcı	Kod
HLT	2

- Halt komutu ile karşılaşıldı

Anımsatıcı	Kod
ADR	3

- Hatalı adres (komut ya da veri) ile karşılaşıldı

Anımsatıcı	Kod
INS	4

- Geçersiz komut ile karşılaşıldı

Beklenen davranış

- Eğer AOK ise, çalışmaya devam et
- Değilse, programı çalışmayı durdur

Y86-64 Kodu yazımı

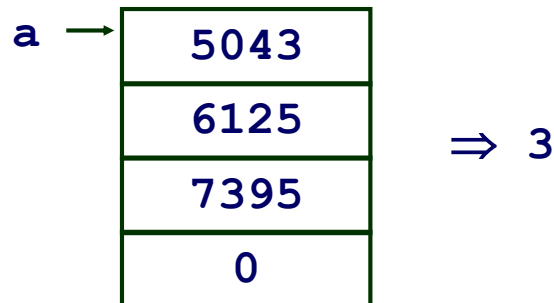
Mümkün olduğunca C derleyiciyi kullanmaya çalış

- Kodu önce C'de yaz
- x86-64 için gcc -Og -S komutuyla derle
- Y86-64 koduna dönüştür
- *Modern derleyiciler bunu oldukça zorlaştırmaktadır.*

Kodlama Örneği

- Null ile sonlandırılmış bir listedeki eleman sayısının bulunması

```
int len1(int a[]);
```



Y86-64 Kodu Oluşturma Örneği

İlk deneme

- Tipik bir dizi kodu yaz

```
/* Null ile sonlandırılmış bir
listedeki eleman sayısının bul
*/
long len(long a[])
{
    long len;
    for (len = 0; a[len]; len++)
        ;
    return len;
}
```

- gcc -Og -S ile derle

Problem

- Dizi indekslemeyi yapmak Y86-64'da zor
 - Çünkü ölçeklendirilmiş adresleme modu bulunmuyor

L3:

```
addq $1,%rax
cmpq $0, (%rdi,%rax,8)
jne L3
```

Y86-64 Kodu Oluşturma Örneği #2

İkinci deneme

- Beklenen Y86-64 koduna benzeyen bir C kodu yaz

```
long len2(long *a)
{
    long ip = (long) a;
    long val = *(long *) ip;
    long len = 0;
    while (val) {
        ip += sizeof(long);
        len++;
        val = *(long *) ip;
    }
    return len;
}
```

Sonuç

- Derleyici ilk denemedekiyle aynı kodu üretir!
- Derleyici her iki versiyonu da aynı ara forma çevirir



Y86-64 Code Generation Example #3

len:

```
irmovq $1, %r8      # Sabit 1
irmovq $8, %r9      # Sabit 8
irmovq $0, %rax     # len = 0
mrmovq (%rdi), %rdx # val = *a
andq %rdx, %rdx     # val'i test et
je Done            # Sıfırsa,
                  # Done'a git
```

Loop:

```
addq %r8, %rax      # len++
addq %r9, %rdi      # a++
mrmovq (%rdi), %rdx # val = *a
andq %rdx, %rdx     # val'i test et
jne Loop           # Sıfır değilse,
                  # Loop'a git
```

Done:

```
ret
```

Kaydedici	Kullanım
%rdi	a
%rax	len
%rdx	val
%r8	1
%r9	8

Y86-64 Örnek Program Yapısı #1

```
init:                                # Başlangıç ayarları
    . . .
    call Main
    halt

    .align 8                          # Program verisi
array:
    . . .

Main:                                # Ana fonksiyon
    . . .
    call len    . . .

len:                                  # Uzunluk fonksiyonu
    . . .

    .pos 0x100                        # Yığın yerleşimi
Stack:
```

- Program 0 adresinden başlar
- Yığının ayarlanması gerekir
 - Nerede bulunacak
 - Pointer değerleri
 - Kodun üzerine yazmadığımızdan emin olmalıyız!
- Verinin ilk değerlerinin ayarlanması gerekir

Y86-64 Örnek Program Yapısı #2

```
init:
    # Yığının işaretçisini ayarla
    irmovq Stack, %rsp
    # main programı çalıştır
    call Main
    # Dur
    halt

# 4 elemanlık Array + 0 ile bitiyor
    .align 8
array:
    .quad 0x000d000d000d000d
    .quad 0x00c000c000c000c0
    .quad 0x0b000b000b000b00
    .quad 0xa000a000a000a000
    .quad 0
```

- Program 0 adresinden başlar
- Yığının ayarlanması gerekir
- Verinin ilk değerlerinin ayarlanması gerekir
- Sembolik isimler kullanılabilir

Y86-64 Örnek Program Yapısı #3

Main:

```
irmovq array,%rdi
# call len(array)
call len
ret
```

len altprogramının çağırılması

- array adresi %rdi üzerinden argüman olarak gönderilmiştir.

Y86-64 Programının Derlenmesi

```
unix> yas len.ys
```

- len.yo adlı “obje kodu” nu oluşturur
 - Aslında ayırıcı (disassembler) çıktısına benzer

```

0x054:          | len:
0x054: 30f8010000000000000000 |   irmovq $1, %r8           # Constant 1
0x05e: 30f9080000000000000000 |   irmovq $8, %r9           # Constant 8
0x068: 30f0000000000000000000 |   irmovq $0, %rax          # len = 0
0x072: 5027000000000000000000 |   mrmovq (%rdi), %rdx       # val = *a
0x07c: 6222          |   andq %rdx, %rdx          # Test val
0x07e: 73a0000000000000000000 |   je Done                  # If zero, goto Done
0x087:          | Loop:
0x087: 6080          |   addq %r8, %rax           # len++
0x089: 6097          |   addq %r9, %rdi           # a++
0x08b: 5027000000000000000000 |   mrmovq (%rdi), %rdx       # val = *a
0x095: 6222          |   andq %rdx, %rdx          # Test val
0x097: 7487000000000000000000 |   jne Loop                 # If !0, goto Loop
0x0a0:          | Done:
0x0a0: 90            |   ret

```



Y86-64 Programının Simüle Edilmesi

```
unix> yis len.yo
```

■ Komut seti simulatorü

- Her bir komutun işlemci durumu üzerindeki etkisini hesaplar
- Orjinal durum üzerinde gerçekleşen değişiklikleri ekrana basar

```
Stopped in 33 steps at PC = 0x13.  Status 'HLT', CC Z=1 S=0 O=0
```

```
Changes to registers:
```

%rax:	0x0000000000000000	0x0000000000000004
%rsp:	0x0000000000000000	0x0000000000000100
%rdi:	0x0000000000000000	0x0000000000000038
%r8:	0x0000000000000000	0x0000000000000001
%r9:	0x0000000000000000	0x0000000000000008

```
Changes to memory:
```

0x00f0:	0x0000000000000000	0x0000000000000053
0x00f8:	0x0000000000000000	0x0000000000000013

2018 Final Çıkmış Soru

Soru 3: (16 puan) Aşağıda verilen Y86-64 assembly kod parçasındaki **prog** alt programı çağrılmadan hemen önce, ana hafıza içeriği şekilde verildiği gibiyse ve **%rdi** kaydedicisinde **0x140** değeri bulunuyorsa, **ret** komutu işletilmeden hemen önce **%rax**, **%rdx**, **%rdi** ve **%r11** kaydedicilerinin içerikleri ne olur? Aşağıda verilen kutucuklara **hexadecimal (onaltılık) şekilde yazınız**. Ana hafızadaki her bir kutu ilgili adreste kayıtlı bulunan **8 byte**'lık veriyi göstermektedir.

prog:

```
irmovq $5, %r10
irmovq $1, %r9
irmovq $8, %r8
irmovq $0, %rax
mrmovq (%rdi), %rdx
rrmovq %rdx, %r11
subq %r10, %rdx
andq %rdx, %rdx
je Son
```

dongu:

```
addq %r11, %rax
subq %r8, %rdi
mrmovq (%rdi), %rdx
rrmovq %rdx, %r11
subq %r10, %rdx
andq %rdx, %rdx
jne dongu
```

Son:

ret

Ana Hafıza

	Adres
6	0x100
3	0x108
5	0x110
8	0x118
5	0x120
7	0x128
0	0x130
2	0x138
9	0x140
7	0x148

%rax	
------	--

%rdx	
------	--

%rdi	
------	--

%r11	
------	--

CISC Komut Setleri

- Karmaşık Komut Setli Bilgisayar (Complex Instruction Set Computer)
- IA32 bir örnektir

Yığına yönelik komut seti

- Yığın argüman aktarmak ve program sayacını kaydetmek için kullanılır
- Doğrudan push ve pop komutları bulunur

Aritmetik komutlar hafızaya erişebilir

- `addq %rax, 12(%rbx,%rcx,8)`
 - Hafızadan okuma ve hafızaya yazma işlemleri gerektirir
 - Karmaşık adres hesabı gerekir

Durum kodları (Condition codes)

- Aritmetik ve lojik komutların yan etkisi olarak değiştirilirler

Felsefesi

- Tipik programlama işlerini gerçekleştirecek komutlar eklemek

RISC Komut Setleri

- Azaltılmış Komut Setli Bilgisayar (Reduced Instruction Set Computer)
- IBM'in bir iç projesiyken, daha sonra Hennessy (Stanford) ve Patterson (Berkeley) tarafından popülerlik kazandırılmıştır

Daha az sayıda ve daha basit komutlar

- Verilen işi tamamlamak için fazla sayıda komut gerekebilir
- Bu komutları daha küçük ve hızlı donanım ile işletebilir

Kaydedicilere yönelik komut seti

- Çok sayıda (tipik olarak 32) kaydedici
- Argümanlar, dönüş adresi ve geçici değişkenler için

Sadece yükle(load) ve kaydet(store) komutları hafızaya erişebilir

- Y86-64'daki `lrmovq` ve `rrmovq` komutlarına benzer şekilde

Durum kodları (Condition codes) bulunmaz

- Test komutları kaydedicide 0/1 döner

CISC ve RISC

Orjinal Tartışma

- CISC taraftarları---derleyici için kolay, daha az kod boyutu
- RISC taraftarları---optimizasyon yapan derleyiciler için daha iyi, basit çip tasarımıyla hızlı çalıştırılabilir

Şimdiki Durum

- Masaüstü işlemcileri için ISA seçimi teknik bir mesele değildir
 - Yeterli donanım ile her şey hızlı çalıştırılabilir
 - Kod uyumluluğu daha önemlidir
- x86-64 birçok RISC özelliğini uygulamaktadır
 - Daha fazla kaydedici; bunların argüman aktarımı için kullanımı
- Gömülü işlemciler için, RISC anlamlıdır
 - Daha küçük, daha ucuz, daha az güç harcayan
 - Çoğu cep telefonu ARM işlemci kullanır

Özet

Y86-64 Komut Seti Mimarisi

- x86-64'dakine benzer durum ve komutlar
- Daha basit kodlama
- CISC ve RISC arasında bir konumda

ISA Tasarımı Ne Kadar Önemlidir?

- Geçmiştekinden daha az
 - Yeterli donanım ile her şey hızlı çalıştırılabilir

2017 Vize / 14. soru

Float sayıların IEEE 754 standardına göre 1 bit işaret biti (s), 5 bit üst (exp) ve 5 bit anlamlı kısım (frac) ile ifade edildiği bir sistemde 1'den büyük en küçük float sayının ikili ifadesi aşağıdakilerden hangisidir? (Şıklardaki bit sırası soldan sağa şu şekildedir: işaret biti, üst ve anlamlı kısım.) (6 puan)

- A. 0 01110 11111
- B. 0 01111 00001
- C. 0 00001 00001
- D. 0 01111 00000

Kaynaklar

- 1. Computer Systems: A Programmer's Perspective, Randal E. Bryant ve David R. O'Hallaron, PEARSON.**

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



TEŞEKKÜRLER